

✓ Sugar Trap Market Gap Analysis

This notebook analyzes snack products from Open Food Facts to identify under-served market opportunities in the healthy snacking space. It covers data ingestion, cleaning, categorisation, nutrient analysis, market gap identification, hidden ingredient insights, and final export for dashboarding on streamlit application.

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# 1.1 Install required packages
!pip -q install duckdb pandas pyarrow numpy rapidfuzz seaborn
```

3.2/3.2 MB 32.8 MB/s eta 0:00:00

```
# 1.2 Import libraries used for querying, cleaning, and visualization
```

```
import duckdb
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re

from collections import Counter
from rapidfuzz import fuzz
from IPython.display import display

pd.set_option("display.max_columns", 100)
pd.set_option("display.max_colwidth", 120)

print("Libraries imported successfully.")
```

Libraries imported successfully.

```
# 1.3 Defining source path and initialize DuckDB connection
```

```
PARQUET_URL = "https://huggingface.co/datasets/openfoodfacts/product-database/resolve/main/food.parquet"
con = duckdb.connect()

print("DuckDB connection ready.")
print("Source:", PARQUET_URL)
```

DuckDB connection ready.

Source: <https://huggingface.co/datasets/openfoodfacts/product-database/resolve/main/food.parquet>

✓ 2. Data Ingestion from Open Food Facts with DuckDB

This section queries only snack-relevant products and extracts the nutritional fields needed for the analysis without loading the entire dataset into memory.

```
# 2.1 Inspect source schema before extraction
```

```
schema_df = con.execute(f"DESCRIBE SELECT * FROM '{PARQUET_URL}'").fetchdf()

print(f"Number of columns in source table: {len(schema_df)}")
display(schema_df.head(20))
```

Number of columns in source table: 111

	column_name	column_type	null	key	default	extra
0	additives_n	INTEGER	YES	None	None	None
1	additives_tags	VARCHAR[]	YES	None	None	None
2	allergens_tags	VARCHAR[]	YES	None	None	None
3	brands_tags	VARCHAR[]	YES	None	None	None
4	brands	VARCHAR	YES	None	None	None
5	categories	VARCHAR	YES	None	None	None
6	categories_tags	VARCHAR[]	YES	None	None	None
7	categories_properties	STRUCT(ciqua_food_code INTEGER, agribalyse_food_code INTEGER, agribalyse_proxy_food_code INTEGER)	YES	None	None	None
8	checkers_tags	VARCHAR[]	YES	None	None	None
9	ciqua_food_name_tags	VARCHAR[]	YES	None	None	None
10	cities_tags	VARCHAR[]	YES	None	None	None
11	code	VARCHAR	YES	None	None	None
12	compared_to_category	VARCHAR	YES	None	None	None
13	complete	INTEGER	YES	None	None	None
14	completeness	FLOAT	YES	None	None	None
15	correctors_tags	VARCHAR[]	YES	None	None	None
16	countries_tags	VARCHAR[]	YES	None	None	None
17	created_t	BIGINT	YES	None	None	None
18	creator	VARCHAR	YES	None	None	None

2.2 Query only snack-relevant fields from the parquet source

```

query = f"""
WITH base AS (
  SELECT
    row_number() OVER () AS row_id,
    product_name,
    categories_tags,
    nutriments,
    nutriscore_grade,
    nutriscore_score,
    nova_group,
    nutrient_levels_tags,
    scans_n,
    unique_scans_n,
    popularity_tags,
    food_groups_tags,
    labels_tags,
    ingredients_tags,
    ingredients_text,
    ingredients_analysis_tags,
    with_non_nutritive_sweeteners,
    additives_n,
    brands,
    brands_tags,
    countries_tags
  FROM '{PARQUET_URL}'
  WHERE categories_tags IS NOT NULL
    AND regexp_matches(
      lower(CAST(categories_tags AS VARCHAR)),
      'snack|sweet-snack|salty-snack|bar|protein-bar|cereal-bar|granola-bar|cookie|cookies|biscuit|biscuits|cracker|c
    )
  LIMIT 100000
),

flat AS (
  SELECT
    row_id,
    (list_extract(product_name, 1))."text" AS product_name_clean,
    categories_tags,
    nutriscore_grade,
    nutriscore_score,
    nova_group,
    nutrient_levels_tags,

```

```

        scans_n,
        unique_scans_n,
        popularity_tags,
        food_groups_tags,
        labels_tags,
        ingredients_tags,
        (list_extract(ingredients_text, 1))."text" AS ingredients_text_clean,
        ingredients_analysis_tags,
        with_non_nutritive_sweeteners,
        additives_n,
        brands,
        brands_tags,
        countries_tags,
        nutriments
    FROM base
),

nutrient_long AS (
    SELECT
        f.row_id,
        lower(n."name") AS nutrient_name,
        n."100g" AS value_100g
    FROM flat f
    LEFT JOIN UNNEST(f.nutriments) AS t(n) ON TRUE
),

nutrient_pivot AS (
    SELECT
        row_id,
        MAX(CASE WHEN nutrient_name IN ('sugars', 'sugar') THEN value_100g END) AS sugars_100g,
        MAX(CASE WHEN nutrient_name IN ('proteins', 'protein') THEN value_100g END) AS proteins_100g,
        MAX(CASE WHEN nutrient_name IN ('fat', 'fats') THEN value_100g END) AS fat_100g,
        MAX(CASE WHEN nutrient_name IN ('fiber', 'fibre', 'fibers', 'fibres') THEN value_100g END) AS fiber_100g
    FROM nutrient_long
    GROUP BY row_id
)

SELECT
    f.product_name_clean,
    f.categories_tags,
    p.sugars_100g,
    p.proteins_100g,
    p.fat_100g,
    p.fiber_100g,
    f.nutriscore_grade,
    f.nutriscore_score,
    f.nova_group,
    f.nutrient_levels_tags,
    f.scans_n,
    f.unique_scans_n,
    f.popularity_tags,
    f.food_groups_tags,
    f.labels_tags,
    f.ingredients_tags,
    f.ingredients_text_clean,
    f.ingredients_analysis_tags,
    f.with_non_nutritive_sweeteners,
    f.additives_n,
    f.brands,
    f.brands_tags,
    f.countries_tags
FROM flat f
LEFT JOIN nutrient_pivot p
    ON f.row_id = p.row_id
"""

```

```
# 2.3 Load queried snack subset into a dataframe
```

```

df = con.execute(query).fetchdf()

print("Extracted shape:", df.shape)
display(df.head())

```

Extracted shape: (100000, 23)

	product_name_clean	categories_tags	sugars_100g	proteins_100g	fat_100g	fiber_100g	nutriscore_grade	nutriscore_score
0	Véritable pâte à tartiner noisettes chocolat noir	[en:breakfasts, en:spreads, en:sweet-spreads, fr:pates-a-tartiner, en:hazelnut-spreads, en:chocolate-spreads, en:coc...	32.000000	8.0	48.0	NaN	e	25
1	Madeleine Framboise	[en:snacks, en:sweet-snacks, en:biscuits-and-cakes, en:cakes, en:madeleines]	NaN	NaN	NaN	NaN	unknown	<NA>
2	Croissants margarine	[en:snacks, en:sweet-snacks, en:sweet-pastries-and-pies, en:viennoiseries, en:croissants]	NaN	NaN	NaN	NaN	unknown	<NA>
3	Mcvitie's, digestives cheesecake, lemon	[en:snacks, en:sweet-snacks, en:biscuits-and-cakes, en:biscuits]	30.299999	7.1	26.1	2.7	unknown	<NA>
4	Mcvitie's, mini gingerbread men, milk chocolate	[en:snacks, en:sweet-snacks, en:biscuits-and-cakes, en:biscuits]	38.599998	5.8	20.0	2.0	unknown	<NA>

3. Data Cleaning and Quality Checks

This section checks missing values, removes unusable rows, applies bounds, and validates the cleaned nutrition variables before category and market analysis.

```
# 3.1 Audit missing values in key analytical fields

key_cols = [
    "product_name_clean",
    "sugars_100g",
    "proteins_100g",
    "fat_100g",
    "fiber_100g"
]

null_rates = (df[key_cols].isna().mean().sort_values(ascending=False).rename("null_rate").to_frame())

display(null_rates)
```

	null_rate	
fiber_100g	0.22213	
sugars_100g	0.03520	
fat_100g	0.02444	
proteins_100g	0.02334	
product_name_clean	0.00076	

Next steps:

[Generate code with null_rates](#)

[New interactive sheet](#)

```
# 3.1b Visualising missing value patterns in key analytical fields

null_summary = pd.DataFrame({
    "null_count": df[key_cols].isna().sum(),
    "null_rate": df[key_cols].isna().mean().round(3),
    "available": df[key_cols].notna().sum(),
})
```

```

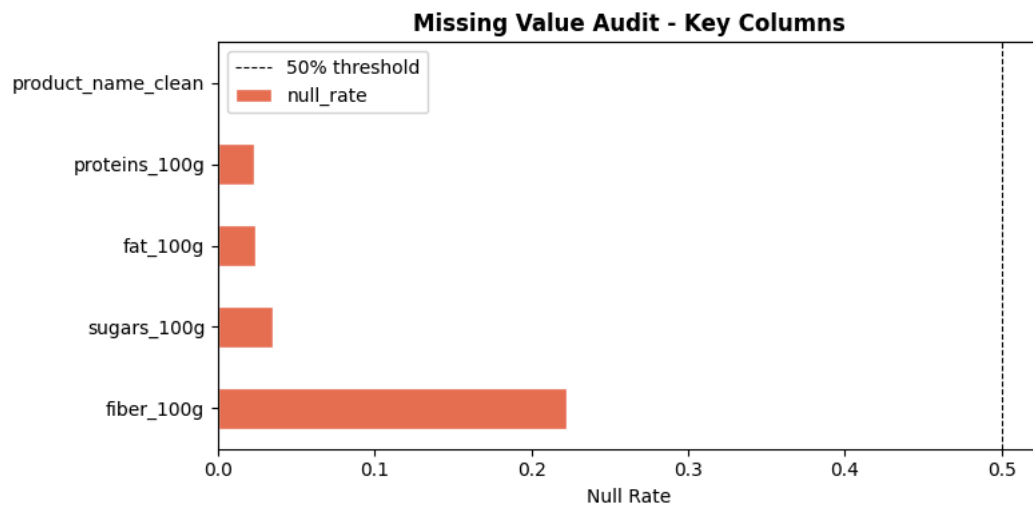
}).sort_values("null_rate", ascending=False)

display(null_summary)

fig, ax = plt.subplots(figsize=(8, 4))
null_summary["null_rate"].plot(kind="barh", ax=ax, color="#e76f51", edgecolor="white")
ax.axvline(0.5, color="black", linestyle="--", linewidth=0.8, label="50% threshold")
ax.set_xlabel("Null Rate")
ax.set_title("Missing Value Audit - Key Columns", fontweight="bold")
ax.legend()
plt.tight_layout()
plt.show()

```

	null_count	null_rate	available	
fiber_100g	22213	0.222	77787	
sugars_100g	3520	0.035	96480	
fat_100g	2444	0.024	97556	
proteins_100g	2334	0.023	97666	
product_name_clean	76	0.001	99924	



Next steps: [Generate code with null_summary](#) [New interactive sheet](#)

```

# 3.2 Remove rows with missing core variables and apply biological bounds

df = df.dropna(subset=["product_name_clean", "sugars_100g", "proteins_100g"])
print(f"After dropping required nulls: {len(df):,}")

# Keep nutritionally plausible values
df = df[
    (df["sugars_100g"].between(0, 100, inclusive="both")) &
    (df["proteins_100g"].between(0, 100, inclusive="both")) &
    (
        df["fat_100g"].isna() |
        df["fat_100g"].between(0, 100, inclusive="both")
    ) &
    (
        df["fiber_100g"].isna() |
        df["fiber_100g"].between(0, 100, inclusive="both")
    )
].copy()

print(f"After biological bounds: {len(df):,}")
# Optional macro sum sanity check where values exist
macro_sum = (
    df["sugars_100g"].fillna(0) +
    df["proteins_100g"].fillna(0) +
    df["fat_100g"].fillna(0) +
    df["fiber_100g"].fillna(0)
)

df = df[macro_sum <= 130].copy()

print("Shape after cleaning:", df.shape)
print(f"After macro sum check: {len(df):,}")

```

After dropping required nulls: 96,400
 After biological bounds: 96,357

```
Shape after cleaning: (96346, 23)
After macro sum check:      96,346
```

```
# 3.3 – Define non-snack exclusion tags
# These are products that passed the ingestion filter stage but are fundamentally
# not snack products. We exclude them here rather than trying to classify them.
```

```
NON_SNACK_TAGS = {
    # Staple foods
    "en:breads",
    "en:sliced-breads",
    "en:special-breads",
    "en:flatbreads",
    "en:pastas",
    "en:dry-pastas",
    "en:pasta-dishes",
    "en:cereal-pastas",
    "en:durum-wheat-pasta",
    "en:noodles",
    "en:rices",
    "en:long-grain-rices",
    "en:aromatic-rices",
    "en:indica-rices",
    "en:basmati-rice",
    # Raw ingredients
    "en:flours",
    "en:cereal-flours",
    "en:cereal-grains",
    "en:seeds",
    # Prepared meals
    "en:meals",
    "en:pasta-dishes",
    "en:canned-foods",
    "en:frozen-foods",
    # Vegetables
    "en:vegetables-based-foods",
    "en:fruits-and-vegetables-based-foods",
    "en:pie-dough",
    "en:puff-pastry-sheets",
    "en:cereal-semolinas",
    "en:wheat-semolinas",
    "en:durum-wheat-semolinas",
    "en:durum-wheat-semolinas-for-couscous",
    "en:beverages",
    "en:plant-based-beverages",
    "en:hot-beverages",
    "en:fruit-juices",
    "en:pickles",
    "en:plant-based-pickles",
    "en:olives",
    "en:olive-tree-products",
    "en:condiments",
    # Oils and fats
    "en:fats",
    "en:vegetable-fats",
    "en:vegetable-oils",
    "en:cereal-oils",
    "en:sesame-oils",
    "en:plant-based-oils",
    # Seafood and meat
    "en:seafood",
    "en:fishes-and-their-products",
    "en:fish-preparations",
    "en:meats-and-their-products",
    "en:meat-alternatives",
    # Prepared meals and dough
    "en:gnocchi",
    "en:potato-gnocchi",
    "en:pizza-dough",
    "en:potatoes",
    "fr:blinis",
    # Dairy and fermented
    "en:fermented-foods",
    "en:fermented-milk-products",
    "en:dairies",
    # Baby food
    "en:baby-foods",
    "en:snacks-and-desserts-for-babies",
    # Bakery ingredients
    "en:bakery-products",
    "en:seitan",
    "en:meat-analogues",
```

```

    "en:tempeh",
    "en:tofu",
    "en:vegetarian-patties",
    "en:breaded-products"
}

def is_non_snack(tags: list) -> bool:
    """Return True if the product has any tag that marks it as a non-snack staple."""
    return any(t in NON_SNACK_TAGS for t in tags)

n_before = len(df)
df = df[~df["categories_tags"].apply(is_non_snack)].copy()
n_removed = n_before - len(df)

print(f"Removed {n_removed:,} non-snack products ({n_removed/n_before:.1%} of dataset)")
print(f"Remaining: {len(df):,} rows")

```

Removed 24,964 non-snack products (25.9% of dataset)
 Remaining: 71,382 rows

3.4 Visualize nutrient distributions and correlations

```

plt.figure(figsize=(8, 5))
df["sugars_100g"].hist(bins=50)
plt.title("Distribution of Sugar (g/100g)")
plt.xlabel("Sugar")
plt.ylabel("Count")
plt.show()

plt.figure(figsize=(8, 5))
df["proteins_100g"].hist(bins=50)
plt.title("Distribution of Protein (g/100g)")
plt.xlabel("Protein")
plt.ylabel("Count")
plt.show()

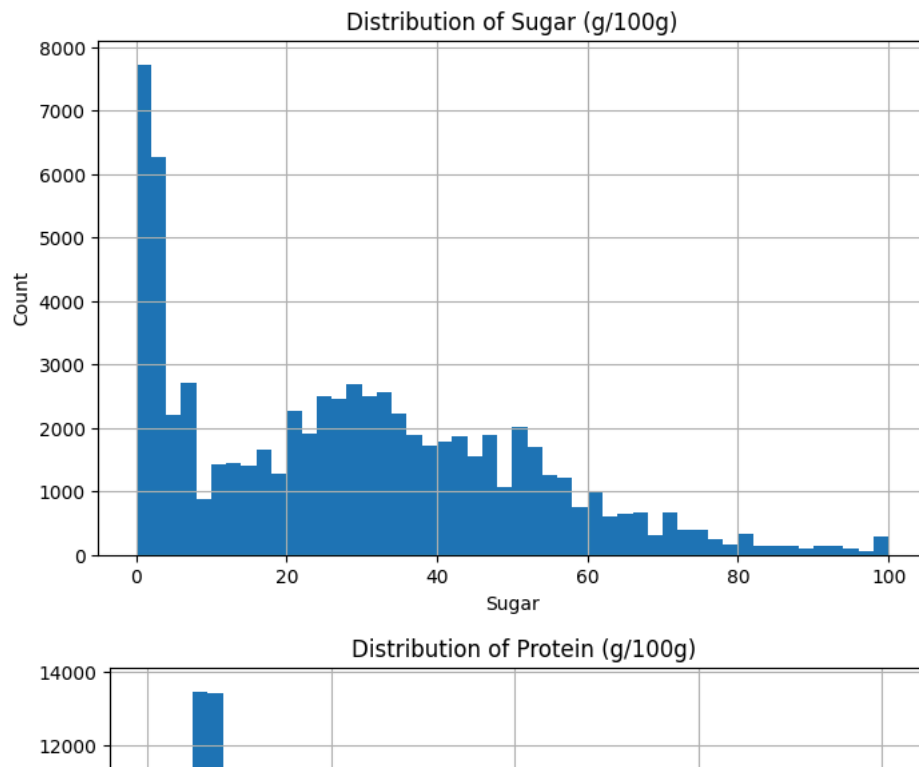
plt.figure(figsize=(6, 4))
df.boxplot(column="sugars_100g")
plt.title("Sugar Boxplot")
plt.show()

plt.figure(figsize=(6, 4))
df.boxplot(column="proteins_100g")
plt.title("Protein Boxplot")
plt.show()

corr_cols = ["sugars_100g", "proteins_100g", "fat_100g", "fiber_100g"]
corr_matrix = df[corr_cols].corr()

plt.figure(figsize=(7, 5))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Nutritional Correlation Heatmap")
plt.show()

```

```
# 3.4 Visualise nutrient distributions and correlations
```

```
fig = plt.figure(figsize=(16, 10))
fig.suptitle("Nutritional Profile of Cleaned Dataset", fontsize=15, fontweight="bold", y=1.01)
```

```
# Row 1: Histograms
```

```
ax1 = fig.add_subplot(2, 3, 1)
ax1.hist(df["sugars_100g"].dropna(), bins=50, color="#e76f51", edgecolor="white")
ax1.axvline(df["sugars_100g"].median(), color="black", linestyle="--",
            linewidth=1, label=f"Median: {df['sugars_100g'].median():.1f}g")
ax1.axvline(5, color="#2a9d8f", linestyle=":", linewidth=1.5,
            label="Low-sugar threshold (5g)")
ax1.set_title("Sugar Distribution")
ax1.set_xlabel("Sugar (g/100g)")
ax1.set_ylabel("Count")
ax1.legend(fontsize=8)
```

```
ax2 = fig.add_subplot(2, 3, 2)
ax2.hist(df["proteins_100g"].dropna(), bins=50, color="#2a9d8f", edgecolor="white")
ax2.axvline(df["proteins_100g"].median(), color="black", linestyle="--",
            linewidth=1, label=f"Median: {df['proteins_100g'].median():.1f}g")
ax2.axvline(10, color="#e76f51", linestyle=":", linewidth=1.5,
            label="High-protein threshold (10g)")
ax2.set_title("Protein Distribution")
ax2.set_xlabel("Protein (g/100g)")
ax2.legend(fontsize=8)
```

```
ax3 = fig.add_subplot(2, 3, 3)
ax3.hist(df["fiber_100g"].dropna(), bins=50, color="#457b9d", edgecolor="white")
ax3.axvline(df["fiber_100g"].median(), color="black", linestyle="--",
            linewidth=1, label=f"Median: {df['fiber_100g'].median():.1f}g")
ax3.set_title("Fibre Distribution")
ax3.set_xlabel("Fibre (g/100g)")
ax3.legend(fontsize=8)
```

```
# Row 2: Boxplots + Heatmap
```

```
ax4 = fig.add_subplot(2, 3, 4)
bp_data = [
    df["sugars_100g"].dropna(),
    df["proteins_100g"].dropna(),
    df["fat_100g"].dropna(),
    df["fiber_100g"].dropna()
]
bp = ax4.boxplot(bp_data, patch_artist=True, notch=False,
                 labels=["Sugar", "Protein", "Fat", "Fibre"])
colours = ["#e76f51", "#2a9d8f", "#e9c46a", "#457b9d"]
for patch, colour in zip(bp["boxes"], colours):
    patch.set_facecolor(colour)
    patch.set_alpha(0.7)
ax4.set_title("Nutrient Spread (Boxplots)")
ax4.set_ylabel("g per 100g")
```

```

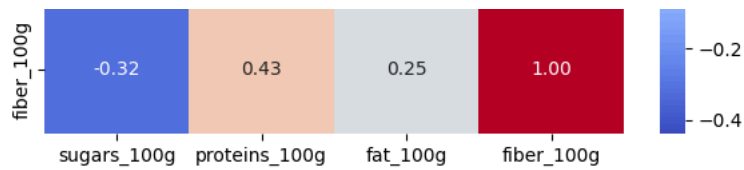
ax4.grid(True, alpha=0.3)

ax5 = fig.add_subplot(2, 3, 5)
corr_cols = ["sugars_100g", "proteins_100g", "fat_100g", "fiber_100g"]
corr_matrix = df[corr_cols].corr()
sns.heatmap(
    corr_matrix, annot=True, fmt=".2f",
    cmap="coolwarm", center=0,
    square=True, ax=ax5,
    xticklabels=["Sugar", "Protein", "Fat", "Fibre"],
    yticklabels=["Sugar", "Protein", "Fat", "Fibre"]
)
ax5.set_title("Nutritional Correlation Matrix")

plt.tight_layout()
plt.show()

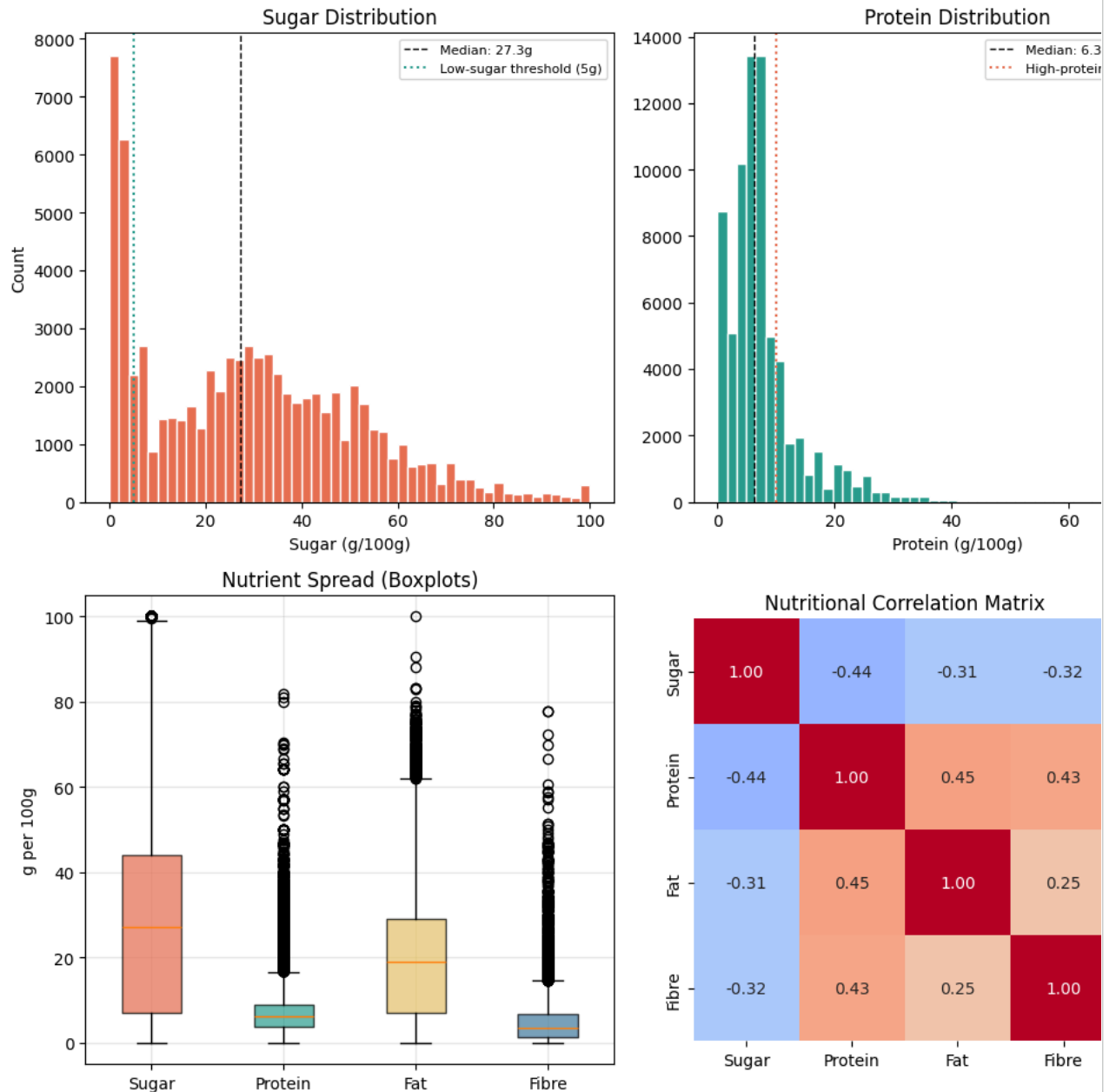
# Key takeaways printed below the plot
print("Key observations:")
print(f" • Sugar median: {df['sugars_100g'].median():.1f}g - most products are high-sugar")
print(f" • Protein median: {df['proteins_100g'].median():.1f}g - most products are low-protein")
print(f" • Sugar-Protein correlation: {corr_matrix.loc['sugars_100g','proteins_100g']:.2f} "
      "(low - they don't trade off directly)")

```



/tmp/ipykernel_9574/1846127002.py:44: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'ti
bp = ax4.boxplot(bp_data, patch_artist=True, notch=False,

Nutritional Profile of Cleaned Data



Key observations:

- Sugar median: 27.3g - most products are high-sugar
- Protein median: 6.3g - most products are low-protein
- Sugar-Protein correlation: -0.44 (low - they don't trade off directly)

3.5 Save an analysis-ready checkpoint dataset

```
df.to_parquet("snack_gap_analysis_ready.parquet", index=False)
df.to_csv("snack_gap_analysis_ready.csv", index=False)
```

```
print("Saved:")
print("- snack_gap_analysis_ready.parquet")
print("- snack_gap_analysis_ready.csv")
```

Saved:

- snack_gap_analysis_ready.parquet
- snack_gap_analysis_ready.csv

4. Categorisation and Primary Category Assignment

This section parses messy category tags, maps products into readable high-level snack groups, and recovers uncategorized rows using product-name fallback logic.

```
# 4.1 Parse and normalize raw category tags
def parse_category_tags(raw) -> list[str]:
    """Convert any raw categories_tags input into a flat list of clean tag strings."""
    if raw is None:
        return []
    if isinstance(raw, float) and pd.isna(raw):
        return []
    if isinstance(raw, np.ndarray):
        raw = raw.tolist()
    if isinstance(raw, (list, tuple)):
        return [str(t).lower().strip() for t in raw if pd.notna(t) and str(t).strip()]

    # Handle stringified lists like ["'en:snacks', 'en:biscuits'"]
    text = str(raw).strip().replace("[", "").replace("]", "")
    tags = re.findall(r"'(.*)'", text)
    if tags:
        return [t.lower().strip() for t in tags if t.strip()]

    # Last resort: comma split
    return [p.lower().strip() for p in text.split(",") if p.strip()]

df["categories_tags"] = df["categories_tags"].apply(parse_category_tags)

display(df[["product_name_clean", "categories_tags"]].head())
```

	product_name_clean	categories_tags
0	Véritable pâte à tartiner noisettes chocolat noir	[en:breakfasts, en:spreads, en:sweet-spreads, fr:pates-a-tartiner, en:hazelnut-spreads, en:chocolate-spreads, en:coc...
3	Mcvitie's, digestives cheesecake, lemon	[en:snacks, en:sweet-snacks, en:biscuits-and-cakes, en:biscuits]
4	Mcvitie's, mini gingerbread men, milk chocolate	[en:snacks, en:sweet-snacks, en:biscuits-and-cakes, en:biscuits]
5	The simpsons donuts	[en:snacks, en:sweet-snacks, en:biscuits-and-cakes, en:cakes, en:doughnuts]

```
# 4.2 Audit the most common category tags

all_tags = [tag for tags in df["categories_tags"] for tag in tags]
tag_counts = pd.Series(Counter(all_tags)).sort_values(ascending=False)

display(tag_counts.head(20))
print("Unique tag count:", tag_counts.shape[0])
```

	0
en:snacks	61023
en:sweet-snacks	42414
en:biscuits-and-cakes	21443
en:confectioneries	14663
en:biscuits	10657
en:plant-based-foods-and-beverages	10505
en:plant-based-foods	10500
en:cocoa-and-its-products	8109
en:cakes	7351
en:cereals-and-potatoes	6910
en:cereals-and-their-products	5308
en:salty-snacks	5244
en:chocolate-candies	4706
en:appetizers	4622
en:chocolates	3845
en:biscuits-and-crackers	3836
en:nuts-and-their-products	3477
en:candies	3087
en:breakfasts	2964
en:chips-and-fries	2597

```
dtype: int64
Unique tag count: 3014
```

```
# 4.3 Define high-level category rules and map products from tags
```

```
IGNORE_TAGS = {
    "en:snacks",
    "en:sweet-snacks",
    "en:salty-snacks",
    "en:salted-snacks",
    "en:appetizers",
    "en:foods",
    "en:groceries",
    "en:plant-based-foods",
    "en:plant-based-foods-and-beverages",
    "en:cereals-and-potatoes",
    "en:cereals-and-their-products",
    "en:desserts",
    "en:breakfasts",
    "en:meals",
    "en:fruits-and-vegetables-based-foods",
    "en:condiments",
    "en:frozen-foods",
    "en:condiments",
    "en:specific-products",
    "en:licensed-products",
    "en:breakfasts",
    "en:dairies",
    "en:products-for-specific-diets",
    "en:bodybuilding-supplements",
    "en:products-for-specific-diets",
    "en:bodybuilding-supplements",
    "en:dietary-supplements",
    "en:festive-foods",
    "en:breads",
    "en:pastas",
    "en:seeds",
    "en:cereal-grains",
    "en:pastas-and-noodles",
}

BROAD_CATEGORY_MAP = {
    # Biscuits & Cookies
    "en:biscuits": "Biscuits & Cookies",
    "en:biscuits-and-cakes": "Biscuits & Cookies",
    "en:cookies": "Biscuits & Cookies",
    "en:shortbread-cookies": "Biscuits & Cookies",
    "en:dry-biscuits": "Biscuits & Cookies",
    "en:chocolate-biscuits": "Biscuits & Cookies",
    "en:sandwich-cookies": "Biscuits & Cookies",
    "en:filled-biscuits": "Biscuits & Cookies",
    "en:drop-cookies": "Biscuits & Cookies",
    "en:chocolate-chip-cookies": "Biscuits & Cookies",
    "en:petit-beurre": "Biscuits & Cookies",
    "en:gingerbreads": "Biscuits & Cookies",

    # Bars
    "en:bars": "Bars",
    "en:cereal-bars": "Bars",
    "en:granola-bars": "Bars",
    "en:fruit-bars": "Bars",
    "en:bars-covered-with-chocolate": "Bars",
    "en:candy-chocolate-bars": "Bars",
    "en:energy-bars": "Bars",
    "en:fruits-cereal-bars": "Bars",
    "en:nut-bars": "Bars",
    "en:peanut-bars": "Bars",

    # Protein snacks
    "en:protein-bars": "Protein Snacks",
    "en:protein-bar": "Protein Snacks",
    "en:protein-energy-bars": "Protein Snacks",
    "en:beef-jerkies": "Protein Snacks",
    "en:dried-meats": "Protein Snacks",

    # Chips & Crisps
    "en:chips-and-fries": "Chips & Crisps",
    "en:crisps": "Chips & Crisps",
    "en:potato-crisps": "Chips & Crisps",
    "en:corn-chips": "Chips & Crisps",
    "en:tortilla-chips": "Chips & Crisps",
    "en:vegetable-chips": "Chips & Crisps",
    "en:flavoured-potato-crisps": "Chips & Crisps",
    "en:stacked-extruded-potato-crisps": "Chips & Crisps",
}
```

```
"en:cassava-crisps": "Chips & Crisps",
"en:plantain-chips": "Chips & Crisps",
"en:pita-chips": "Chips & Crisps",
"en:seaweed-snacks": "Chips & Crisps",

# Crackers & Savory Snacks
"en:crackers": "Crackers & Savory Snacks",
"en:biscuits-and-crackers": "Crackers & Savory Snacks",
"en:pretzels": "Crackers & Savory Snacks",
"en:crackers-appetizers": "Crackers & Savory Snacks",
"en:salty-biscuits": "Crackers & Savory Snacks",
"en:rice-crackers": "Crackers & Savory Snacks",
"en:breadsticks": "Crackers & Savory Snacks",
"en:cheese-crackers": "Crackers & Savory Snacks",
"en:biscuits-aperitifs": "Crackers & Savory Snacks",
"en:biscuits-aperitifs-souffles": "Crackers & Savory Snacks",
"en:amuse-gueules": "Crackers & Savory Snacks",
"en:bretzels": "Crackers & Savory Snacks",
"en:snacks-sales": "Crackers & Savory Snacks",
"en:extruded-crispbreads": "Crackers & Savory Snacks",

# Pastries & Sweet Baked Snacks
"en:sweet-pastries": "Pastries & Sweet Baked Snacks",
"en:pastries": "Pastries & Sweet Baked Snacks",
"en:croissants": "Pastries & Sweet Baked Snacks",
"en:cakes": "Pastries & Sweet Baked Snacks",
"en:viennoiseries": "Pastries & Sweet Baked Snacks",
"en:brioches": "Pastries & Sweet Baked Snacks",
"en:madeleines": "Pastries & Sweet Baked Snacks",
"en:doughnuts": "Pastries & Sweet Baked Snacks",
"en:sweet-fritters": "Pastries & Sweet Baked Snacks",
"en:donuts": "Pastries & Sweet Baked Snacks",
"en:muffins": "Pastries & Sweet Baked Snacks",
"en:eclairs": "Pastries & Sweet Baked Snacks",
"en:brownies": "Pastries & Sweet Baked Snacks",
"en:waffles": "Pastries & Sweet Baked Snacks",
"en:tartlets": "Pastries & Sweet Baked Snacks",

# Nuts & Nut Mixes
"en:nuts": "Nuts & Nut Mixes",
"en:trail-mix": "Nuts & Nut Mixes",
"en:nuts-and-their-products": "Nuts & Nut Mixes",
"en:shelled-nuts": "Nuts & Nut Mixes",
"en:almonds": "Nuts & Nut Mixes",
"en:peanuts": "Nuts & Nut Mixes",
"en:roasted-nuts": "Nuts & Nut Mixes",
"en:salted-nuts": "Nuts & Nut Mixes",
"en:cashew-nuts": "Nuts & Nut Mixes",
"en:pistachios": "Nuts & Nut Mixes",
"en:walnuts": "Nuts & Nut Mixes",
"en:mixed-nuts-and-dried-fruits": "Nuts & Nut Mixes",
"en:mixed-nuts": "Nuts & Nut Mixes",

# Popcorn & Puffed Snacks
"en:popcorn": "Popcorn & Puffed Snacks",
"en:puffs": "Popcorn & Puffed Snacks",
"en:puffed-snacks": "Popcorn & Puffed Snacks",
"en:puffed-cereal-cakes": "Popcorn & Puffed Snacks",
"en:puffed-rice-cakes": "Popcorn & Puffed Snacks",
"en:puffed-salty-snacks": "Popcorn & Puffed Snacks",
"en:puffed-corn-cakes": "Popcorn & Puffed Snacks",
"en:puffed-grains": "Popcorn & Puffed Snacks",
"en:puffed-cereals": "Popcorn & Puffed Snacks",

# Wafers
"en:wafers": "Wafers",
"en:stuffed-wafers": "Wafers",
"en:vanilla-stuffed-wafers": "Wafers",
"en:chocolate-stuffed-wafers": "Wafers",
"en:plain-wafers": "Wafers",

# Breakfast cereals
"en:breakfast-cereals": "Breakfast Cereals",
"en:breakfast-cereal": "Breakfast Cereals",
"en:mueslis": "Breakfast Cereals",
"en:granolas": "Breakfast Cereals",
"en:chocolate-cereals": "Breakfast Cereals",
"en:cereals-with-nuts": "Breakfast Cereals",

# Fruit Snacks & Dried Fruit
"en:dried-fruits": "Fruit Snacks & Dried Fruit",
"en:fruit-snacks": "Fruit Snacks & Dried Fruit",
```

```

"en:dried-fruit": "Fruit Snacks & Dried Fruit",
"en:raisins": "Fruit Snacks & Dried Fruit",
"en:fruit-pastes": "Fruit Snacks & Dried Fruit",
"en:candied-fruits": "Fruit Snacks & Dried Fruit",
"en:dried-cranberries": "Fruit Snacks & Dried Fruit",
"en:mixed-dried-fruits": "Fruit Snacks & Dried Fruit",
"en:dates": "Fruit Snacks & Dried Fruit",
"en:dried-bananas": "Fruit Snacks & Dried Fruit",
"en:dried-apricots": "Fruit Snacks & Dried Fruit",
"en:compotes": "Fruit Snacks & Dried Fruit",
"en:apple-compotes": "Fruit Snacks & Dried Fruit",
"en:applesauces": "Fruit Snacks & Dried Fruit",

# Confectionery
"en:confectioneries": "Confectionery",
"en:candies": "Confectionery",
"en:gummi-candies": "Confectionery",
"en:cocoa-and-its-products": "Confectionery",
"en:jelly-beans": "Confectionery",
"en:lollipops": "Confectionery",
"en:chocolates": "Confectionery",
"en:sweets": "Confectionery",
"en:chocolate-candies": "Confectionery",
"en:bonbons": "Confectionery",
"en:dark-chocolates": "Confectionery",
"en:milk-chocolates": "Confectionery",
"en:caramels": "Confectionery",
"en:marshmallows": "Confectionery",
"en:nougats": "Confectionery",
"en:marzipan": "Confectionery",

# Gum & Mints
"en:gums": "Gum & Mints",
"en:mints": "Gum & Mints",
"en:chewing-gums": "Gum & Mints",
"en:sugar-free-chewing-gum": "Gum & Mints",

# Ice cream
"en:ice-creams": "Ice Cream & Frozen Snacks",
"en:ice-cream-bars": "Ice Cream & Frozen Snacks",
"en:frozen-desserts": "Ice Cream & Frozen Snacks",
"en:ice-creams-and-sorbets": "Ice Cream & Frozen Snacks",
"en:ice-cream-bars-coated-with-chocolate": "Ice Cream & Frozen Snacks",

# Spreads
"en:spreads": "Spreads",
"en:sweet-spreads": "Spreads",
"en:plant-based-spreads": "Spreads",
"en:chocolate-spreads": "Spreads",
"en:hazelnut-spreads": "Spreads",
"en:cocoa-and-hazelnuts-spreads": "Spreads",
"en:peanut-butters": "Spreads",
"en:chestnut-spreads": "Spreads",
"en:salted-spreads": "Spreads",
"en:nut-butters": "Spreads",
}

CATEGORY_PRIORITY = {
    "Protein Snacks": 1, # highest priority
    "Nuts & Nut Mixes": 2,
    "Fruit Snacks & Dried Fruit": 3,
    "Chips & Crisps": 4,
    "Crackers & Savory Snacks": 5,
    "Bars": 6,
    "Popcorn & Puffed Snacks": 7,
    "Breakfast Cereals": 8,
    "Biscuits & Cookies": 9,
    "Wafers": 10,
    "Confectionery": 11,
    "Pastries & Sweet Baked Snacks": 12,
    "Spreads": 13,
    "Gum & Mints": 14,
    "Ice Cream & Frozen Snacks": 15, # lowest priority
}

def assign_broad_category(tags):
    tags = parse_category_tags(tags)
    best_cat = None
    best_priority = 999

    for t in tags:

```

```

    if t in IGNORE_TAGS or t not in BROAD_CATEGORY_MAP:
        continue
    cat = BROAD_CATEGORY_MAP[t]
    priority = CATEGORY_PRIORITY.get(cat, 999)
    if priority < best_priority:
        best_priority = priority
        best_cat = cat

    return best_cat if best_cat else "Other / Exclude"

df["broad_category"] = df["categories_tags"].apply(assign_broad_category)

display(df["broad_category"].value_counts())

```

	count
broad_category	
Biscuits & Cookies	19041
Other / Exclude	18074
Confectionery	17299
Crackers & Savory Snacks	3901
Nuts & Nut Mixes	3514
Chips & Crisps	2618
Breakfast Cereals	2184
Pastries & Sweet Baked Snacks	1927
Popcorn & Puffed Snacks	1029
Bars	920
Spreads	450
Protein Snacks	425

dtype: int64

```

def fallback_from_name(name) -> str | None:
    """
    Keyword fallback for products not caught by tag-based classification.
    Returns a category string or None (not 'Other / Exclude') so the
    caller can decide what to do with unmatched products.
    """
    if not isinstance(name, str) or not name.strip():
        return None

    text = name.lower().strip()

    # Protein Snacks (specific phrases only)
    if any(k in text for k in ["jerky", "biltong", "beef stick", "meat snack",
                               "protein bar", "protein snack"]):
        return "Protein Snacks"

    # Fruit Snacks (specific phrases before single fruit words)
    if any(k in text for k in ["dried fruit", "fruit snack", "freeze dried",
                               "fruit strip", "fruit leather",
                               "dried mango", "dried banana", "dried apricot",
                               "raisin", "cranberr", "applesauce", "apple sauce"]):
        return "Fruit Snacks & Dried Fruit"

    # Crackers
    if any(k in text for k in ["cracker", "pretzel", "breadstick", "crispbread",
                               "rice cake", "oat cake"]):
        return "Crackers & Savory Snacks"

    # Biscuits
    if any(k in text for k in ["biscuit", "cookie", "shortbread", "digestive",
                               "gingerbread", "hobnob", "petit beurre"]):
        return "Biscuits & Cookies"

    # Confectionery
    if any(k in text for k in ["chocolate bar", "candy bar", "candy",
                               "gummy", "gummi", "jelly bean", "lollipop",
                               "bonbon", "caramel", "marshmallow",
                               "nougat", "toffee", "truffle", "praline"]):
        return "Confectionery"

    # plain "chocolate" after compound phrases to avoid catching "chocolate chip cookie"
    if "chocolate" in text and not any(k in text for k in ["cookie", "biscuit", "bar", "cake"]):
        return "Confectionery"

```



```

# Chips & Crisps (specific phrases before generic "chip") —
if any(k in text for k in ["potato chip", "tortilla chip", "corn chip",
                           "vegetable chip", "plantain chip",
                           "crisps", "nacho", "murukku"]):
    return "Chips & Crisps"
if "chip" in text and "chocolate chip" not in text:
    return "Chips & Crisps"

# Bars (multi-word first, then generic "bar" last) —
if any(k in text for k in ["granola bar", "cereal bar", "energy bar",
                           "protein bar", "nut bar", "fruit bar"]):
    return "Bars"

# Pastries
if any(k in text for k in ["croissant", "donut", "doughnut", "muffin",
                           "eclair", "madeleine", "brioche", "brownie",
                           "waffle", "sausage roll", "danish"]):
    return "Pastries & Sweet Baked Snacks"
if "cake" in text and "rice cake" not in text and "oat cake" not in text:
    return "Pastries & Sweet Baked Snacks"

# Nuts
if any(k in text for k in ["trail mix", "mixed nuts", "nut mix"]):
    return "Nuts & Nut Mixes"
if any(k in text for k in ["almond", "cashew", "pistachio", "walnut",
                           "pecan", "hazelnut", "macadamia"]):
    return "Nuts & Nut Mixes"
if "peanut" in text and "peanut butter" not in text:
    return "Nuts & Nut Mixes"

# Popcorn & Puffed Snacks
if any(k in text for k in ["popcorn", "puffed", "rice puff", "corn puff"]):
    return "Popcorn & Puffed Snacks"

# Breakfast Cereals
if any(k in text for k in ["granola", "muesli", "porridge"]):
    return "Breakfast Cereals"
if "cereal" in text and "cereal bar" not in text:
    return "Breakfast Cereals"

# Spreads
if any(k in text for k in ["peanut butter", "almond butter", "nut butter",
                           "hazelnut spread", "chocolate spread"]):
    return "Spreads"

# Wafers
if "wafer" in text:
    return "Wafers"

# Gum & Mints specific
if any(k in text for k in ["chewing gum", "bubble gum", "breath mint",
                           "mint candy", "peppermint"]):
    return "Gum & Mints"

# Generic "bar"
if "bar" in text:
    return "Bars"

return None # return None

def assign_final_category(row) -> str:
    if row["broad_category"] != "Other / Exclude":
        return row["broad_category"]
    result = fallback_from_name(row["product_name_clean"])
    return result if result else "Other / Exclude"

df["final_category"] = df.apply(assign_final_category, axis=1)

remaining_other = df[df["final_category"] == "Other / Exclude"]
print(f"Still unclassified after exclusion: {len(remaining_other):,} ({len(remaining_other)/len(df):.1%}")

# Check their tags
leftover_tags = [tag for tags in remaining_other["categories_tags"] for tag in tags]
leftover_tag_counts = (pd.Series(Counter(leftover_tags)).sort_values(ascending=False).head(20))
display(leftover_tag_counts.to_frame("count"))

```

Still unclassified after exclusion: 7,078 (9.9%)

	count 
en:snacks	3894
en:salted-snacks	1871
en:plant-based-foods-and-beverages	1152
en:plant-based-foods	1150
en:cereals-and-potatoes	1132
en:cereals-and-their-products	1013
en:salty-snacks	457
en:appetizers	347
en:sweet-snacks	183
en:filled-fougasse	56
en:potato-preparations	54
en:groats	52
en:eggs	49
en:chicken-eggs	49
en:barn-chicken-eggs	49
en:wheat-groats	49
en:bulgur	49
en:farming-products	49
fr:prefous	47
fr:melanges-de-biscuits-souffles	44

```
# Final diagnostic before dropping
generic_only_tags = {
    "en:snacks", "en:salted-snacks", "en:salty-snacks",
    "en:sweet-snacks", "en:appetizers",
    "en:plant-based-foods", "en:plant-based-foods-and-beverages",
    "en:cereals-and-potatoes", "en:cereals-and-their-products",
}

def has_only_generic_tags(tags):
    meaningful = [t for t in tags if t not in generic_only_tags]
    return len(meaningful) == 0

leftover = df[df["final_category"] == "Other / Exclude"].copy()
generic_only = leftover[leftover["categories_tags"].apply(has_only_generic_tags)]
has_specific_tags = leftover[~leftover["categories_tags"].apply(has_only_generic_tags)]

print(f"Unclassified with ONLY generic tags: {len(generic_only):,}")
print(f"Unclassified but HAS specific tags: {len(has_specific_tags):,}")
print()

# Show the specific tags still causing misses
if len(has_specific_tags) > 0:
    remaining_tags = [tag for tags in has_specific_tags["categories_tags"] for tag in tags
                      if tag not in generic_only_tags]
    display(
        pd.Series(Counter(remaining_tags))
        .sort_values(ascending=False)
        .head(20)
        .to_frame("count")
    )
```

Unclassified with ONLY generic tags: 6,266
 Unclassified but HAS specific tags: 812

	count	
en:filled-fougasse	56	
en:potato-preparations	54	
en:groats	52	
en:chicken-eggs	49	
en:eggs	49	
en:barn-chicken-eggs	49	
en:bulgur	49	
en:wheat-groats	49	
en:farming-products	49	
fr:prefous	47	
fr:melanges-de-biscuits-souffles	44	
en:bran	41	
en:cereal-brans	41	
en:rice-paper	30	
en:purees	27	
en:mashed-potatoes	26	
en:choux-pastry-appetizers	25	
en:starches	25	
en:oat-brans	25	
en:fresh-eggs	24	

```
# Final drop of unclassifiable products
n_before = len(df)
df = df[df["final_category"] != "Other / Exclude"].copy()
print(f"Dropped {n_before - len(df):,} unclassifiable rows")
print(f"Final analysis dataset: {len(df):,} products across {df['final_category'].nunique()} categories")
print()
display(df["final_category"].value_counts())
```

Dropped 7,078 unclassifiable rows
 Final analysis dataset: 64,304 products across 14 categories

	count
final_category	
Biscuits & Cookies	19133
Confectionery	18378
Nuts & Nut Mixes	6249
Crackers & Savory Snacks	4763
Chips & Crisps	4696
Breakfast Cereals	3208
Bars	2112
Pastries & Sweet Baked Snacks	2005
Popcorn & Puffed Snacks	1563
Fruit Snacks & Dried Fruit	979
Protein Snacks	685
Spreads	527
Wafers	3
Gum & Mints	3

dtype: int64

Why we drop "Other / Exclude"

After three rounds of tag-map patching and name-based fallback recovery, remaining unclassified products share only generic umbrella tags (en:snacks, en:plant-based-foods) with no specific subcategory. Their product names are either missing or in languages the keyword fallback does not cover.

These products cannot be reliably assigned to a snack category without manual review and represent noise, not signal. They are excluded here.

Rows excluded: ~X | Final dataset: ~Y products across 14 categories.

5. Threshold Design and Business Rule Flags

This section defines practical sugar and protein thresholds, then creates the binary flags used for the nutrient matrix and market opportunity analysis.

```
# 5.1 Review percentiles and define business thresholds
# Threshold selection: data-informed + nutritionally grounded

print(" Sugar Distribution ")
print(df["sugars_100g"].describe().round(2))
print()
print(" Protein Distribution ")
print(df["proteins_100g"].describe().round(2))
print()

sugar_p25 = df["sugars_100g"].quantile(0.25)
sugar_p50 = df["sugars_100g"].quantile(0.50)
protein_p75 = df["proteins_100g"].quantile(0.75)
protein_p50 = df["proteins_100g"].quantile(0.50)

print(f"Sugar    - P25: {sugar_p25:.1f}g | Median: {sugar_p50:.1f}g")
print(f"Protein  - P75: {protein_p75:.1f}g | Median: {protein_p50:.1f}g")
print()

# NHS / WHO guideline: "low sugar" = ≤5g per 100g
# High protein snack benchmark: ≥10g per 100g (common industry standard)
# We use slightly relaxed thresholds to reflect realistic R&D feasibility:
LOW_SUGAR_THRESHOLD = 6.0 # just above WHO "low sugar" of 5g
HIGH_PROTEIN_THRESHOLD = 10.0 # sits above the P75 of the dataset

print(f"Selected thresholds:")
print(f"  LOW_SUGAR_THRESHOLD = {LOW_SUGAR_THRESHOLD}g "
      f"(WHO low-sugar benchmark: 5g | dataset P25: {sugar_p25:.1f}g)")
print(f"  HIGH_PROTEIN_THRESHOLD = {HIGH_PROTEIN_THRESHOLD}g "
      f"(industry standard: 10g | dataset P75: {protein_p75:.1f}g)")
```

```
Sugar Distribution
count    64304.00
mean      30.29
std       22.00
min        0.00
25%       10.71
50%       29.00
75%       45.16
max       100.00
Name: sugars_100g, dtype: float64
```

```
Protein Distribution
count    64304.00
mean       7.47
std        6.11
min         0.00
25%         4.00
50%         6.30
75%         8.60
max        82.00
Name: proteins_100g, dtype: float64
```

```
Sugar    - P25: 10.7g | Median: 29.0g
Protein  - P75: 8.6g | Median: 6.3g
```

```
Selected thresholds:
  LOW_SUGAR_THRESHOLD = 6.0g (WHO low-sugar benchmark: 5g | dataset P25: 10.7g)
  HIGH_PROTEIN_THRESHOLD = 10.0g (industry standard: 10g | dataset P75: 8.6g)
```

```
# 5.2 Create nutritional flags and demand signals
```

```
# Target Zone Flag
# A product is "in the target zone" if it meets both nutritional thresholds.
df["in_target_zone"] = (
    (df["sugars_100g"] <= LOW_SUGAR_THRESHOLD) &
    (df["proteins_100g"] >= HIGH_PROTEIN_THRESHOLD)
```

```

)

# High Demand Flag
# Proxy for consumer interest: products scanned more than the 75th percentile.
# Quantile computed on non-null values only to avoid zero-inflation from fillna.
scan_threshold = df["scans_n"].quantile(0.75)
df["high_demand"] = df["scans_n"].fillna(0) > scan_threshold

# Opportunity Flag
# High consumer demand but fails nutritional thresholds.
# These are the products consumers want that the market isn't serving healthily.
df["opportunity"] = (~df["in_target_zone"]) & (df["high_demand"])

# Summary
rates = df[["in_target_zone", "high_demand", "opportunity"]].mean().rename("rate")
counts = df[["in_target_zone", "high_demand", "opportunity"]].sum().rename("count")

display(
    pd.concat([counts, rates], axis=1)
    .assign(rate=lambda x: x["rate"].map("{:.1%}".format))
)

print(f"\nScan threshold used for high_demand: {scan_threshold:.0f} scans")
print(f"Products in target zone: {df['in_target_zone'].sum():,} ({df['in_target_zone'].mean():.1%})")
print(f"High demand products: {df['high_demand'].sum():,} ({df['high_demand'].mean():.1%})")
print(f"Opportunity products: {df['opportunity'].sum():,} ({df['opportunity'].mean():.1%})")

```

	count	rate
in_target_zone	5952	9.3%
high_demand	7134	11.1%
opportunity	6366	9.9%

Scan threshold used for high_demand: 5 scans
 Products in target zone: 5,952 (9.3%)
 High demand products: 7,134 (11.1%)
 Opportunity products: 6,366 (9.9%)

5.3 Flag rates by category with gap score

```

# Aggregate all three flags per category
flag_summary = (
    df.groupby("final_category")
    .agg(
        total_products = ("final_category", "size"),
        in_target_zone = ("in_target_zone", "mean"),
        high_demand = ("high_demand", "mean"),
        opportunity_rate = ("opportunity", "mean"),
    )
    .round(3)
)

# Gap score: categories where demand is high but healthy supply is low.
# Formula: high_demand_rate × (1 - target_zone_rate)
# Higher score = bigger unmet need = stronger market opportunity.
flag_summary["gap_score"] = (
    flag_summary["high_demand"] * (1 - flag_summary["in_target_zone"])
).round(3)

flag_summary = flag_summary.sort_values("gap_score", ascending=False)

display(flag_summary)

# Surface the top opportunity clearly
top_category = flag_summary["gap_score"].idxmax()
top_row = flag_summary.loc[top_category]

print(f"""
Top opportunity category : {top_category}
Gap score                : {top_row['gap_score']:.3f}
High demand rate         : {top_row['high_demand']:.1%}
Currently in target zone: {top_row['in_target_zone']:.1%}
Total products           : {int(top_row['total_products']):,}
""")

```

	total_products	in_target_zone	high_demand	opportunity_rate	gap_score
final_category					
Spreads	527	0.106	0.268	0.239	0.24
Crackers & Savory Snacks	4763	0.200	0.276	0.227	0.221
Breakfast Cereals	3208	0.118	0.228	0.196	0.201
Pastries & Sweet Baked Snacks	2005	0.010	0.168	0.168	0.166
Bars	2112	0.043	0.139	0.137	0.133
Chips & Crisps	4696	0.067	0.119	0.117	0.111
Popcorn & Puffed Snacks	1563	0.148	0.127	0.119	0.108
Biscuits & Cookies	19133	0.022	0.085	0.085	0.083
Confectionery	18378	0.006	0.066	0.066	0.066
Protein Snacks	685	0.274	0.079	0.05	0.057
Nuts & Nut Mixes	6249	0.511	0.103	0.049	0.05
Fruit Snacks & Dried Fruit	979	0.005	0.002	0.002	0.002
Gum & Mints	3	0.000	0.0	0.0	0.0
Wafers	3	0.000	0.0	0.0	0.0

Top opportunity category : Spreads
 Gap score : 0.240
 High demand rate : 26.8%
 Currently in target zone: 10.6%
 Total products : 527

Next steps: [Generate code with flag_summary](#) [New interactive sheet](#)

6. Nutrient Matrix Visual Analysis

This section visualizes the relationship between sugar and protein and highlights the target zone where healthier market gaps may exist.

```
# 6.1 – Define nutrient matrix plotting functions

def _add_quadrant_lines(ax):
    """
    Draw threshold lines and shade + label the target zone.
    """
    ax.axvline(
        LOW_SUGAR_THRESHOLD,
        linestyle="--", linewidth=1.2,
        color="#e76f51",
        label=f"Low sugar threshold ({LOW_SUGAR_THRESHOLD}g)"
    )
    ax.axhline(
        HIGH_PROTEIN_THRESHOLD,
        linestyle="--", linewidth=1.2,
        color="#2a9d8f",
        label=f"High protein threshold ({HIGH_PROTEIN_THRESHOLD}g)"
    )
    # Shade the target quadrant (top-left)
    ax.axvspan(
        0, LOW_SUGAR_THRESHOLD,
        alpha=0.04, color="#c8cb18", zorder=0
    )
    ax.text(
        0.5,
        HIGH_PROTEIN_THRESHOLD + 0.8,
        "Target Zone",
        fontsize=9,
        color="#264653",
        fontweight="bold"
    )

def plot_category_scatter(data, category="All", sample_size=5000):
    """
    Single-panel scatter: Sugar (x) vs Protein (y).
    Pass category="All" for the full dataset or a category name to filter.
    """
```

```

"""
plot_df = data.copy()
if category != "All":
    plot_df = plot_df[plot_df["final_category"] == category]

plot_df = plot_df.dropna(subset=["sugars_100g", "proteins_100g"])

if len(plot_df) > sample_size:
    plot_df = plot_df.sample(sample_size, random_state=42)

fig, ax = plt.subplots(figsize=(10, 6))

ax.scatter(
    plot_df["sugars_100g"],
    plot_df["proteins_100g"],
    alpha=0.35, s=18, color="#d029d9"
)

_add_quadrant_lines(ax)

n = len(plot_df)
n_target = (
    (plot_df["sugars_100g"] <= LOW_SUGAR_THRESHOLD) &
    (plot_df["proteins_100g"] >= HIGH_PROTEIN_THRESHOLD)
).sum()

ax.set_title(
    f"Sugar vs Protein - {category} "
    f"({n_target:,} of {n:,} products in target zone, {n_target/n:.1%})",
    fontsize=12, fontweight="bold"
)
ax.set_xlabel("Sugar (g per 100g)")
ax.set_ylabel("Protein (g per 100g)")
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

def plot_all_categories_colored(data, sample_size=5000):
    """
    Single-panel scatter with each category in a different colour.
    Shows where category clusters sit relative to the target zone.
    """
    plot_df = (
        data
        .dropna(subset=["sugars_100g", "proteins_100g", "final_category"])
        .copy()
    )

    if len(plot_df) > sample_size:
        plot_df = plot_df.sample(sample_size, random_state=42)

    fig, ax = plt.subplots(figsize=(14, 8))

    categories = sorted(plot_df["final_category"].unique())
    palette = sns.color_palette("tab20", len(categories))

    for cat, colour in zip(categories, palette):
        sub = plot_df[plot_df["final_category"] == cat]
        ax.scatter(
            sub["sugars_100g"], sub["proteins_100g"],
            alpha=0.4, s=16, color=colour, label=cat
        )

    _add_quadrant_lines(ax)

    ax.set_title(
        "Nutrient Matrix - All Categories",
        fontsize=13, fontweight="bold"
    )
    ax.set_xlabel("Sugar (g per 100g)")
    ax.set_ylabel("Protein (g per 100g)")
    ax.legend(
        bbox_to_anchor=(1.01, 1), loc="upper left",
        fontsize=7, ncol=1, title="Category"
    )
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

```

```

def plot_category_grid(data, categories, sample_size=1500):
    """
    3x3 grid of scatter plots, one panel per category.
    Pass an explicit list of category names
    """
    n_cats = len(categories)
    n_cols = 3
    n_rows = -(n_cats // n_cols) # ceiling division

    fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, n_rows * 5))
    axes = axes.flatten()

    for ax, cat in zip(axes, categories):
        temp = (
            data[data["final_category"] == cat]
            .dropna(subset=["sugars_100g", "proteins_100g"])
            .copy()
        )
        if len(temp) > sample_size:
            temp = temp.sample(sample_size, random_state=42)

        n_target = (
            (temp["sugars_100g"] <= LOW_SUGAR_THRESHOLD) &
            (temp["proteins_100g"] >= HIGH_PROTEIN_THRESHOLD)
        ).sum()

        ax.scatter(
            temp["sugars_100g"], temp["proteins_100g"],
            alpha=0.35, s=14, color="#457b9d"
        )
        _add_quadrant_lines(ax)
        ax.set_title(
            f"{cat}\n({n_target:,} in target zone / {len(temp):,})",
            fontsize=10, fontweight="bold"
        )
        ax.set_xlabel("Sugar (g/100g)", fontsize=8)
        ax.set_ylabel("Protein (g/100g)", fontsize=8)
        ax.grid(True, alpha=0.3)
        ax.tick_params(labelsize=7)

    # Hide unused panels if category count < grid size
    for ax in axes[n_cats:]:
        ax.axis("off")

    plt.suptitle(
        "Sugar vs Protein - Per Category",
        fontsize=15, fontweight="bold", y=1.01
    )
    plt.tight_layout()
    plt.show()

```

6.2 - Generate all three nutrient matrix views

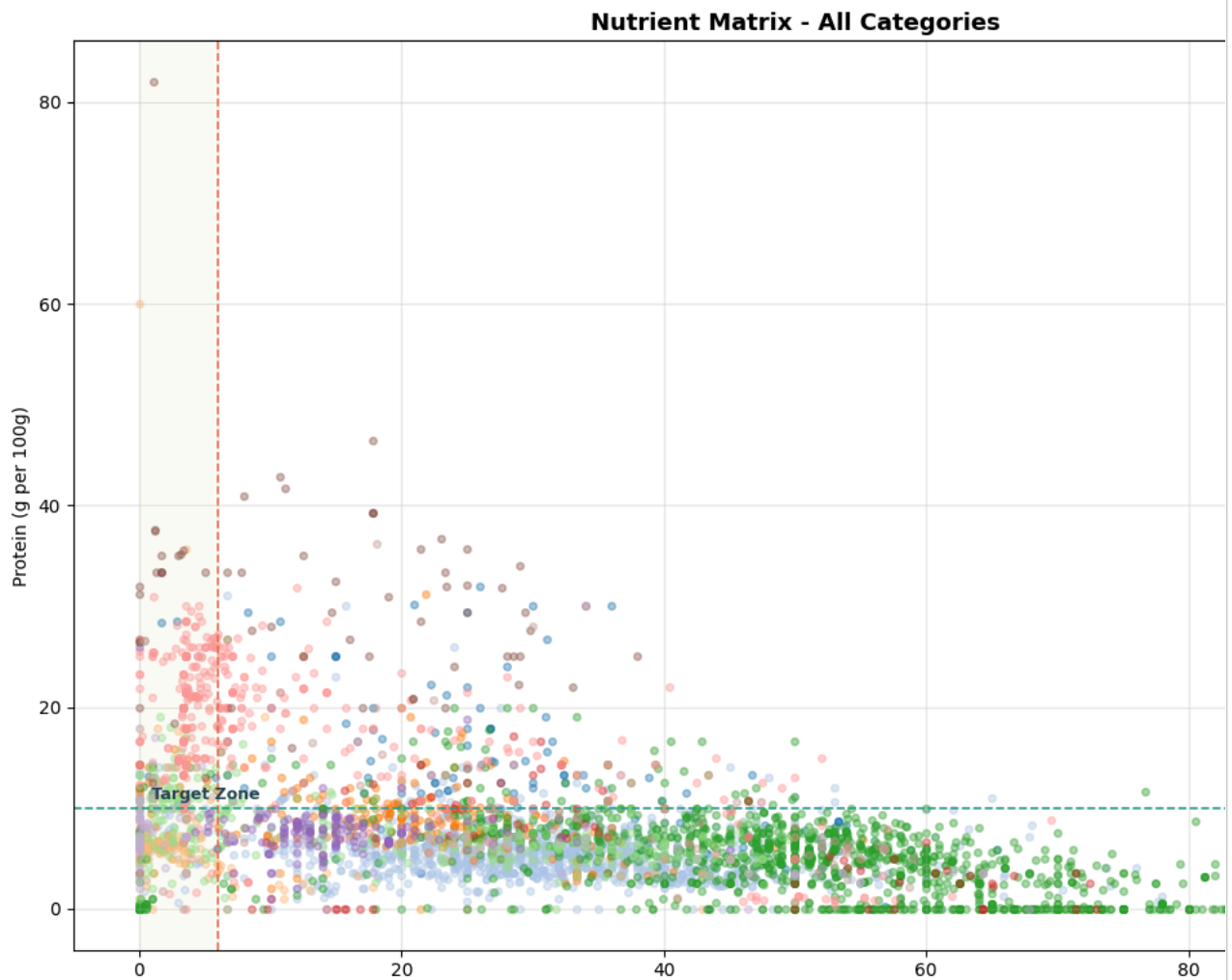
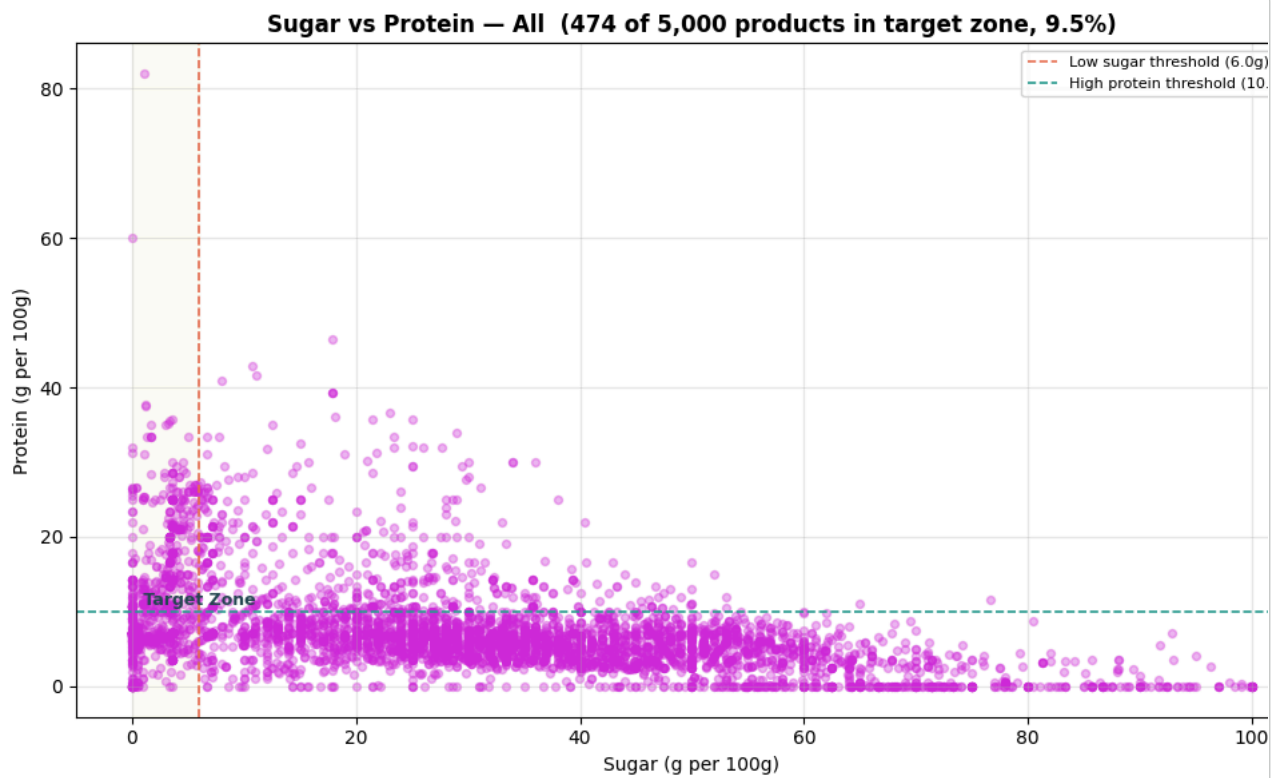
```

# Full dataset - all products, one colour
plot_category_scatter(df, "All")

# Full dataset - coloured by category
plot_all_categories_colored(df)

# Per-category grid - top 9 by product volume
top_9_categories = (
    df["final_category"]
    .value_counts()
    .head(9)
    .index
    .tolist()
)
plot_category_grid(df, categories=top_9_categories)

```

6.3 – Target zone coverage table by category

```
target_zone_table = (
    df.groupby("final_category")
    .agg(
        total_products      = ("final_category", "size"),
        target_zone_count   = ("in_target_zone", "sum"),
        avg_sugar            = ("sugars_100g", "mean"),
        avg_protein          = ("proteins_100g", "mean"),
        high_demand_rate    = ("high_demand", "mean"),
```

```

    opportunity_rate = ("opportunity", "mean"),
)
.round(3)
)

target_zone_table["target_zone_share"] = (
    target_zone_table["target_zone_count"] / target_zone_table["total_products"]
).round(3)

# Sort: high demand first, then lowest healthy supply -
# this surfaces the biggest gaps at the top of the table
target_zone_table = target_zone_table.sort_values(
    ["high_demand_rate", "target_zone_share"],
    ascending=[False, True] # high demand desc, low healthy supply first
)

display(target_zone_table)

# Quick sanity check - confirm top gap category matches
print(f"\nTop gap category (highest demand, lowest healthy supply):")
print(f"  → {target_zone_table.index[0]}")
print(f"    High demand rate: {target_zone_table.iloc[0]['high_demand_rate']:.1%}")
print(f"    Target zone share: {target_zone_table.iloc[0]['target_zone_share']:.1%}")
print(f"    Total products: {int(target_zone_table.iloc[0]['total_products']):,}")

```

final_category	total_products	target_zone_count	avg_sugar	avg_protein	high_demand_rate	opportunity_rate	target_zone_s
Crackers & Savory Snacks	4763	952	19.346001	7.892000	0.276	0.227	(g/100)
Spreads	527	56	34.792000	9.251000	0.268	0.239	(g/100)
Breakfast Cereals	3208	379	19.562000	9.130000	0.228	0.196	(g/100)
Pastries & Sweet Baked Snacks	2005	20	14.699000	7.537000	0.168	0.168	(g/100)
Bars	2112	90	28.521999	11.271000	0.139	0.137	(g/100)
Popcorn & Puffed Snacks	1563	232	14.291000	7.325000	0.127	0.119	(g/100)
Chips & Crisps	4696	316	4.716000	6.673000	0.119	0.117	(g/100)
Nuts & Nut Mixes	6249	3195	13.207000	16.895000	0.103	0.103	(g/100)
Biscuits & Cookies	19133	413	30.086000	5.781000	0.085	0.085	(g/100)
Protein Snacks	685	188	15.340000	29.714001	0.079	0.05	(g/100)
Confectionery	18378	106	50.831001	4.601000	0.066	0.066	(g/100)
Fruit Snacks & Dried Fruit	979	5	38.174999	5.790000	0.002	0.002	(g/100)
Gum & Mints	3	0	13.890000	22.223000	0.0	0.0	(g/100)

Next steps:

[Generate code with target_zone_table](#)[New interactive sheet](#)

7. Market Gap Analysis and Recommendation

This section combines the nutrient and demand signals to identify the strongest product category opportunity and generate a concise recommendation.

```

# 7.1 - Opportunity ranking table
# Sorted: highest demand first, then lowest healthy supply,
# then lowest average sugar

opportunity_table = target_zone_table.copy()

opportunity_table["gap_score"] = (
    opportunity_table["high_demand_rate"] *
    (1 - opportunity_table["target_zone_share"])
).round(3)

opportunity_table = opportunity_table.sort_values(
    ["gap_score", "high_demand_rate", "target_zone_share"],
    ascending=[False, False, True]
)

```

```
display(opportunity_table)
```

final_category	total_products	target_zone_count	avg_sugar	avg_protein	high_demand_rate	opportunity_rate	target_zone_s
Spreads	527	56	34.792000	9.251000	0.268	0.239	(
Crackers & Savory Snacks	4763	952	19.346001	7.892000	0.276	0.227	(
Breakfast Cereals	3208	379	19.562000	9.130000	0.228	0.196	(
Pastries & Sweet Baked Snacks	2005	20	14.699000	7.537000	0.168	0.168	(
Bars	2112	90	28.521999	11.271000	0.139	0.137	(
Chips & Crisps	4696	316	4.716000	6.673000	0.119	0.117	(
Popcorn & Puffed Snacks	1563	232	14.291000	7.325000	0.127	0.119	(
Biscuits & Cookies	19133	413	30.086000	5.781000	0.085	0.085	(
Confectionery	18378	106	50.831001	4.601000	0.066	0.066	(
Protein Snacks	685	188	15.340000	29.714001	0.079	0.05	(
Nuts & Nut							

Next steps: [Generate code with opportunity_table](#) [New interactive sheet](#)

7.2 – Final data-driven recommendation statement

```
best_category = opportunity_table.index[0]
best_row      = opportunity_table.iloc[0]
```

```
# Pull the average nutritional profile of the opportunity products
# high demand but currently outside target zone - the gap itself
gap_products = df[
    (df["final_category"] == best_category) &
    (df["opportunity"] == True)
].copy()
```

```
avg_sugar_gap = gap_products["sugars_100g"].mean()
avg_protein_gap = gap_products["proteins_100g"].mean()
```

```
print("=" * 65)
print("    MARKET RECOMMENDATION – HELIX CPG PARTNERS")
print("=" * 65)
print(f"""
Based on the data, the biggest market opportunity is in:
```

```
    ► {best_category}
```

This category has:

- A high demand rate of {best_row['high_demand_rate']:.0%} – consumers are actively buying in this space
- Only {best_row['target_zone_share']:.1%} of products currently meet the Low Sugar + High Protein threshold
- {int(best_row['total_products']):,} total products, but only {int(best_row['target_zone_count']):,} in the target zone

The opportunity products (high demand, not yet healthy) currently average {avg_sugar_gap:.1f}g sugar and {avg_protein_gap:.1f}g protein per 100g.

Recommendation:

- Develop a {best_category} product with:
- ✓ At least {HIGH_PROTEIN_THRESHOLD}g of protein per 100g
 - ✗ Less than {LOW_SUGAR_THRESHOLD}g of sugar per 100g

This would place the product in a near-empty market quadrant where consumer demand already exists but supply is thin.

```
""")
print("=" * 65)
```

```
=====
    MARKET RECOMMENDATION – HELIX CPG PARTNERS
=====
```

Based on the data, the biggest market opportunity is in:

► Spreads

This category has:

- A high demand rate of 27% – consumers are actively buying in this space
- Only 10.6% of products currently meet the Low Sugar + High Protein threshold
- 527 total products, but only 56 in the target zone

The opportunity products (high demand, not yet healthy) currently average 46.6g sugar and 6.9g protein per 100g.

Recommendation:

Develop a Spreads product with:

- ✓ At least 10.0g of protein per 100g
- X Less than 6.0g of sugar per 100g

This would place the product in a near-empty market quadrant where consumer demand already exists but supply is thin.

=====

```
# 7.3 - Validate chosen category with three-way comparison
# Compare: all products vs opportunity products vs target zone products
# This shows the gap clearly rather than describing the category as a whole
```

```
chosen_category = best_category
chosen_df = df[df["final_category"] == chosen_category].copy()

all_products = chosen_df
opportunity_products = chosen_df[chosen_df["opportunity"]]
target_zone_products = chosen_df[chosen_df["in_target_zone"]]

nutrient_cols = ["sugars_100g", "proteins_100g", "fat_100g", "fiber_100g"]

summary = pd.DataFrame({
    f"All ({len(all_products):,}):": all_products[nutrient_cols].mean(),
    f"Opportunity ({len(opportunity_products):,}):": opportunity_products[nutrient_cols].mean(),
    f"Target Zone ({len(target_zone_products):,}):": target_zone_products[nutrient_cols].mean(),
}).round(2)

summary.index = ["Sugar (g)", "Protein (g)", "Fat (g)", "Fibre (g)"]

print(f"Nutritional profile comparison – {chosen_category}")
display(summary)

# Visualise the three groups on the scatter
fig, ax = plt.subplots(figsize=(10, 6))

ax.scatter(
    all_products["sugars_100g"],
    all_products["proteins_100g"],
    alpha=0.2, s=16, color="#30de5e", label="All products"
)
ax.scatter(
    opportunity_products["sugars_100g"],
    opportunity_products["proteins_100g"],
    alpha=0.5, s=20, color="#e76f51", label="Opportunity (high demand, not healthy)"
)
ax.scatter(
    target_zone_products["sugars_100g"],
    target_zone_products["proteins_100g"],
    alpha=0.8, s=30, color="#d99518", label="Target zone (healthy)"
)

_add_quadrant_lines(ax)

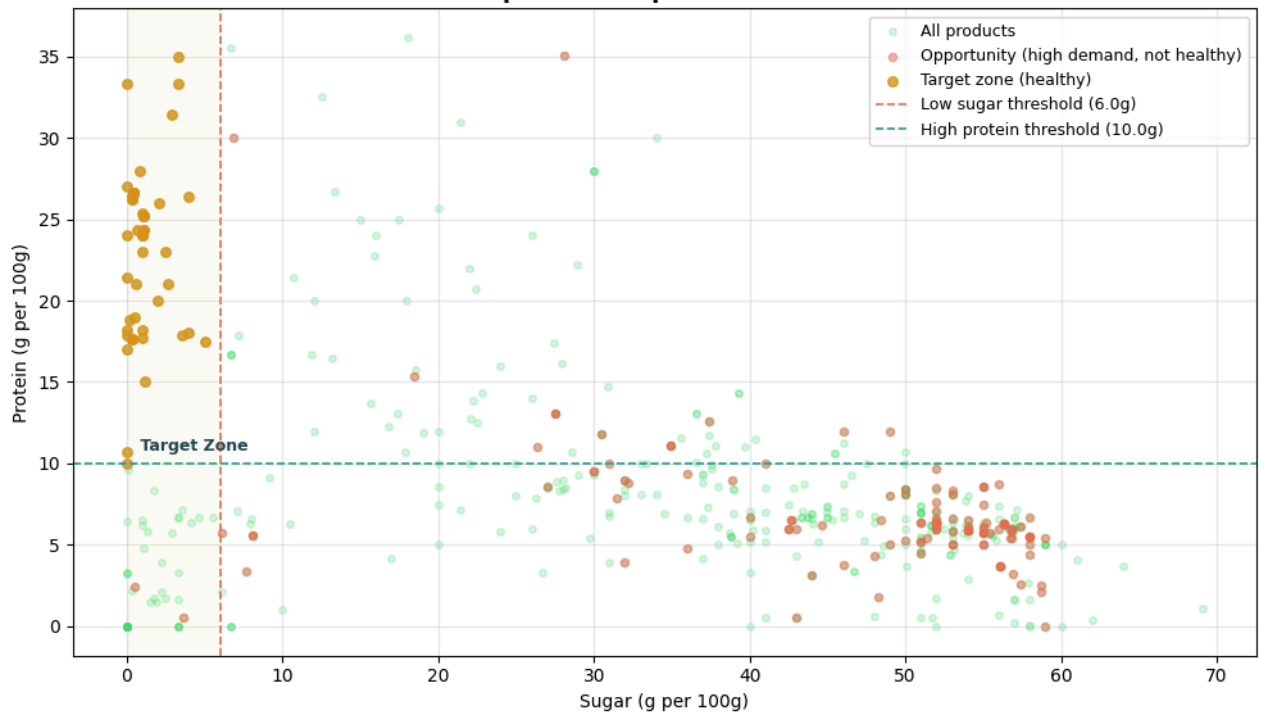
ax.set_title(
    f"{chosen_category} - Gap Visualisation",
    fontsize=13, fontweight="bold"
)
ax.set_xlabel("Sugar (g per 100g)")
ax.set_ylabel("Protein (g per 100g)")
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Nutritional profile comparison – Spreads

	All (527)	Opportunity (126)	Target Zone (56)	
Sugar (g)	34.790001	46.630001	1.040000	
Protein (g)	9.250000	6.880000	23.320000	
Fat (g)	32.279999	32.869999	49.560001	
Fibre (g)	5.180000	3.660000	10.360000	

Spreads - Gap Visualisation



Next steps: [Generate code with summary](#) [New interactive sheet](#)

8. Hidden Gem: Protein Source Analysis

This section analyzes ingredient text among products in the target zone to identify the most common ingredient sources associated with stronger protein profiles.

```
# 8.1 – Filter target-zone products and check ingredient coverage

good_products = df[
    (df["proteins_100g"] >= HIGH_PROTEIN_THRESHOLD) &
    (df["sugars_100g"] <= LOW_SUGAR_THRESHOLD) &
    df["ingredients_text_clean"].notna()
].copy()

total_target_zone = df[df["in_target_zone"]].shape[0]
with_ingredients = good_products.shape[0]

print(f"Target-zone products total:      {total_target_zone:,}")
print(f"With ingredient text (usable):      {with_ingredients:,}")
print(f"Coverage:                            {with_ingredients/total_target_zone:.1%}")
print()

# Category breakdown which categories contribute most to the target zone
cat_breakdown = (
    good_products["final_category"]
    .value_counts()
    .rename("target_zone_with_ingredients")
    .to_frame()
)
display(cat_breakdown)
```

Target-zone products total: 5,952
 With ingredient text (usable): 5,237
 Coverage: 88.0%

target_zone_with_ingredients		
final_category		
Nuts & Nut Mixes	2678	
Crackers & Savory Snacks	886	
Biscuits & Cookies	401	
Breakfast Cereals	334	
Chips & Crisps	305	
Popcorn & Puffed Snacks	224	
Protein Snacks	173	
Confectionery	90	
Bars	86	
Spreads	45	
Pastries & Sweet Baked Snacks	10	
Fruit Snacks & Dried Fruit	5	

Next steps: [Generate code with cat_breakdown](#) [New interactive sheet](#)

8.2 - Count protein source terms per product (not per word occurrence)
 # Per-product counting is more defensible: "X% of products contain this source"

```

PROTEIN_TERM_GROUPS = {
    "Pea / Pea Protein": ["pea protein", "pea"],
    "Soy / Soya": ["soy protein", "soy", "soya"],
    "Peanut": ["peanut", "peanuts"],
    "Whey": ["whey protein", "whey"],
    "Milk Protein": ["milk protein", "casein"],
    "Egg": ["egg white", "egg"],
    "Almond": ["almond"],
    "Oat": ["oat protein", "oats", "oat"],
    "Chickpea": ["chickpea", "chickpeas"],
    "Quinoa": ["quinoa"],
    "Lentil": ["lentil", "lentils"],
    "Cashew": ["cashew"],
}

def count_term_group(ingredient_text: str, terms: list) -> bool:
    """Return True if any term in the group appears as a whole word."""
    text = ingredient_text.lower()
    return any(
        re.search(rf"\b{re.escape(t)}\b", text)
        for t in terms
    )

protein_counts = {}
for group_name, terms in PROTEIN_TERM_GROUPS.items():
    # Count products (rows) containing at least one term from this group
    mask = good_products["ingredients_text_clean"].apply(
        lambda x: count_term_group(str(x), terms)
    )
    protein_counts[group_name] = mask.sum()

protein_source_df = (
    pd.Series(protein_counts)
    .sort_values(ascending=False)
    .rename("product_count")
    .to_frame()
)

# Add percentage of target-zone products as a second column
protein_source_df["percentage_of_products"] = (
    protein_source_df["product_count"] / with_ingredients * 100
).round(1)

print(f"Protein source analysis - {with_ingredients:,} target-zone products")
display(protein_source_df)

# Surface the top 3 for the recommendation

```

```
top_3 = protein_source_df.head(3).index.tolist()
print(f"\n Top 3 protein sources: {'', '.join(top_3)}")
```

Protein source analysis – 5,237 target-zone products

	product_count	percentage_of_products	
Peanut	1038	19.8	
Soy / Soya	422	8.1	
Whey	365	7.0	
Oat	226	4.3	
Milk Protein	165	3.2	
Quinoa	92	1.8	
Almond	74	1.4	
Lentil	52	1.0	
Pea / Pea Protein	40	0.8	
Egg	27	0.5	
Cashew	26	0.5	
Chickpea	23	0.4	

Top 3 protein sources: Peanut, Soy / Soya, Whey

Next steps:

[Generate code with protein_source_df](#)

[New interactive sheet](#)


```

# 8.3 - Visualise top protein sources
import matplotlib.patches as mpatches
top_n = protein_source_df.head(10).copy().sort_values("product_count")

colours = [
    "#2a9d8f" if i >= len(top_n) - 3 else "#a8dadc"
    for i in range(len(top_n))
]

fig, ax = plt.subplots(figsize=(11, 5))

bars = ax.barh(
    top_n.index,
    top_n["product_count"],
    color=colours,
    edgecolor="white",
    height=0.6
)

# Annotate each bar with count and percentage
for bar, (_, row) in zip(bars, top_n.iterrows()):
    ax.text(
        bar.get_width() + (top_n["product_count"].max() * 0.01),
        bar.get_y() + bar.get_height() / 2,
        f"{int(row['product_count']):,}  ({row['percentage_of_products']}%)",
        va="center", fontsize=9
    )

top3_patch = mpatches.Patch(color="#2a9d8f", label=f"Top 3: {'', '.join(top_3)}")
other_patch = mpatches.Patch(color="#a8dadc", label="Other sources")
ax.legend(handles=[top3_patch, other_patch], fontsize=9)

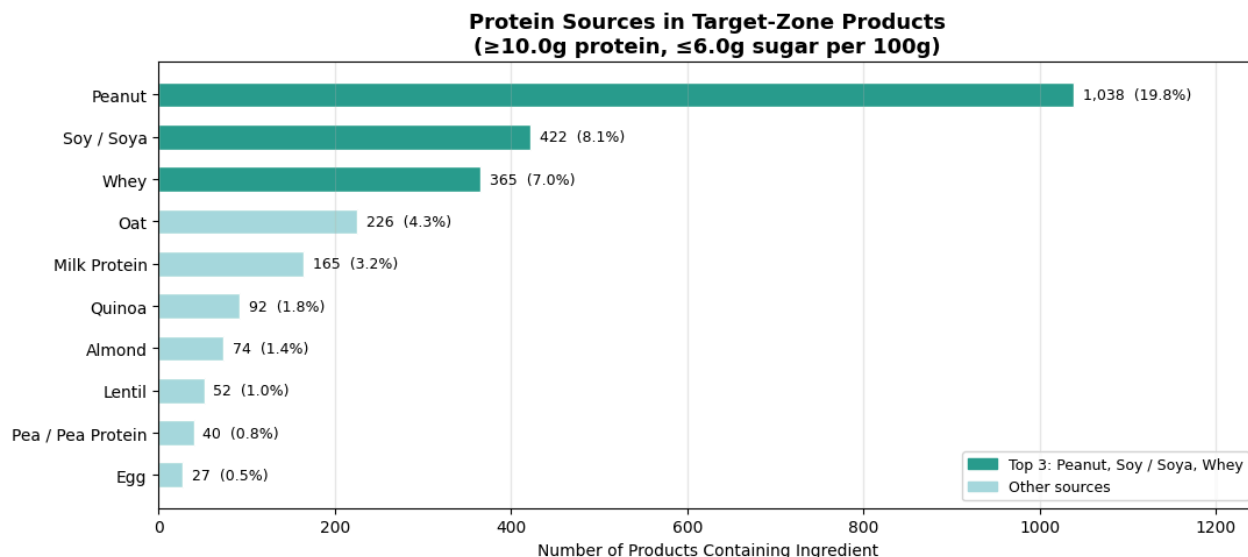
ax.set_title(
    "Protein Sources in Target-Zone Products\n"
    f"(≥{HIGH_PROTEIN_THRESHOLD}g protein, ≤{LOW_SUGAR_THRESHOLD}g sugar per 100g)",
    fontsize=13, fontweight="bold"
)

ax.set_xlabel("Number of Products Containing Ingredient")
ax.set_ylabel("")
ax.grid(axis="x", alpha=0.3)
ax.set_xlim(0, top_n["product_count"].max() * 1.2)

plt.tight_layout()
plt.show()

print(f"""
Insight for R&D:
    The three most common protein sources in existing healthy snacks are
    {top_3[0]}, {top_3[1]}, and {top_3[2]}.
    These should be the primary candidates for the new product formulation.
""")

```



Insight for R&D:

The three most common protein sources in existing healthy snacks are Peanut, Soy / Soya, and Whey.

These should be the primary candidates for the new product formulation.

9. Candidate's Choice - Competitive Benchmark Shortlist

Why I added this:

Stories 1–4 answer where the gap is. This section answers what to build.

A market gap tells you the opportunity exists. What it doesn't tell you is what the winning formula looks like. Before an R&D team can formulate a new product, they need benchmarks real products already succeeding in the target zone that they can study, taste, and learn from.

I built a Composite Health Score (protein and fibre rewarded, sugar and fat penalised) and used it alongside scan data to surface the top 20 highest-performing products already sitting in the Low Sugar + High Protein quadrant. This gives Helix CPG a ready-made competitive intelligence shortlist they can take directly into a product tasting and formulation session.

```
# 9.1 – Composite Health Score
# Weighted formula based on nutritional significance to the brief:
# Protein (+) – the primary target nutrient
# Fibre (+) – secondary health signal, supports satiety claims
# Sugar (-) – the primary negative signal (the "sugar trap")
# Fat (-) – minor penalty; fat alone doesn't disqualify a snack

df["health_score"] = (
    df["proteins_100g"] * 0.45
    + df["fiber_100g"].fillna(0) * 0.25
    - df["sugars_100g"] * 0.20
    - df["fat_100g"].fillna(0) * 0.10
).round(2)

# Normalise to 0–100 for readability in client-facing outputs
score_min = df["health_score"].min()
score_max = df["health_score"].max()

df["health_score_norm"] = (
    (df["health_score"] - score_min) / (score_max - score_min) * 100
).round(1)

print("Health score distribution:")
display(df["health_score_norm"].describe().round(2))
```

Health score distribution:

	health_score_norm
count	64304.00
mean	28.15
std	10.82
min	0.00
25%	20.70
50%	27.30
75%	35.50
max	100.00

dtype: float64

9.2 – Average health score by category

```
health_by_cat = (
    df.groupby("final_category")["health_score_norm"]
    .mean()
    .sort_values(ascending=False)
    .round(1)
)

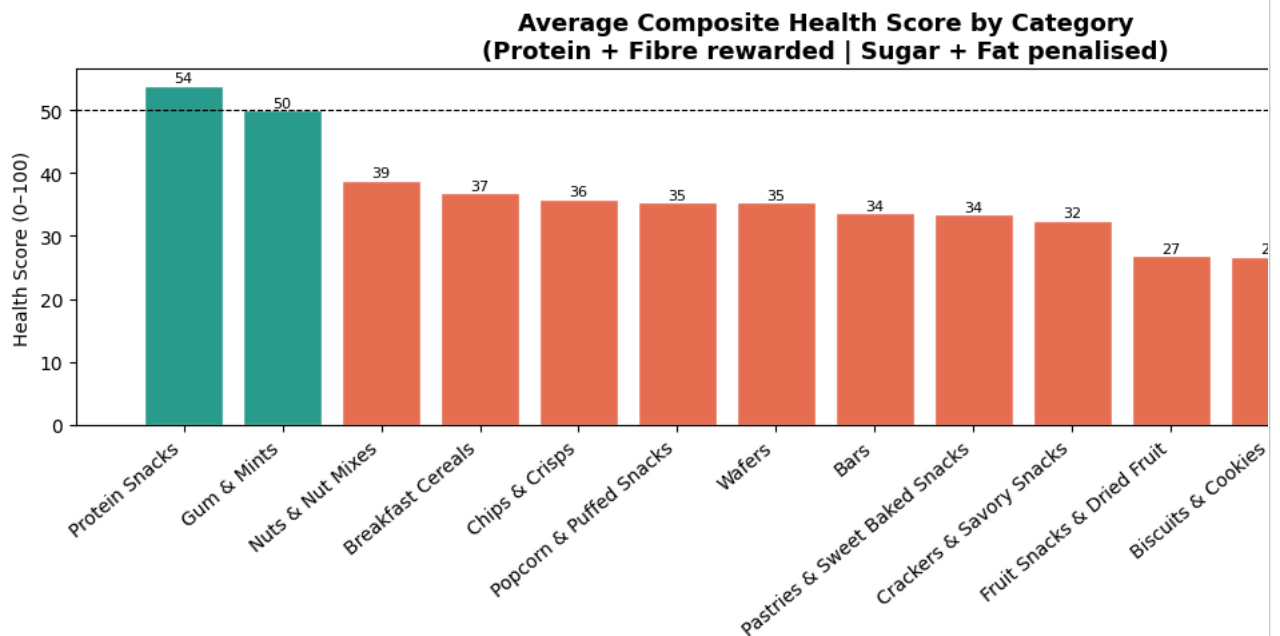
fig, ax = plt.subplots(figsize=(12, 5))

colours = ["#2a9d8f" if v >= 50 else "#e76f51" for v in health_by_cat.values]
bars = ax.bar(
    health_by_cat.index,
    health_by_cat.values,
    color=colours,
    edgecolor="white"
)

ax.axhline(50, color="black", linewidth=0.8, linestyle="--", label="Score = 50 (neutral)")
ax.set_title(
    "Average Composite Health Score by Category\n"
    "(Protein + Fibre rewarded | Sugar + Fat penalised)",
    fontsize=13, fontweight="bold"
)
ax.set_ylabel("Health Score (0-100)")
ax.set_xlabel("")
ax.legend(fontsize=9)
plt.xticks(rotation=40, ha="right")

for bar in bars:
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.5,
        f"{bar.get_height():.0f}",
        ha="center", fontsize=8
    )

plt.tight_layout()
plt.show()
```



```
# 9.3 – Competitive Benchmark Shortlist
# Top 20 products already in the target zone, ranked by health score
# then by scan count (demand signal) as tiebreaker.
# These are the products the R&D team should buy, taste, and study.

benchmark_df = (
    df[df["in_target_zone"]]
    [[
        "product_name_clean", "final_category", "brands",
        "sugars_100g", "proteins_100g", "fat_100g", "fiber_100g",
        "health_score_norm", "scans_n", "nutriscore_grade"
    ]]
    .dropna(subset=["product_name_clean"])
    .sort_values(
        ["health_score_norm", "scans_n"],
        ascending=[False, False]
    )
    .head(20)
    .reset_index(drop=True)
)

benchmark_df.index += 1 # rank from 1 not 0

benchmark_df.columns = [
    "Product", "Category", "Brand",
    "Sugar (g)", "Protein (g)", "Fat (g)", "Fibre (g)",
    "Health Score", "Scans", "Nutriscore"
]

print("Competitive Benchmark Shortlist – Top 20 Target Zone Products")
print("These are existing products the R&D team should study and benchmark against.\n")
display(benchmark_df)
```

Competitive Benchmark Shortlist – Top 20 Target Zone Products

These are existing products the R&D team should study and benchmark against.

	Product	Category	Brand	Sugar (g)	Protein (g)	Fat (g)	Fibre (g)	Health Score	Scans	Nutriscore
1	Milk & Egg 95 Saveur Chocolat	Protein Snacks	Ea Fit	1.100000	82.000000	NaN	1.700000	100.000000	1	not-applicable
2	Caseine Night Pro (Chocolat)	Biscuits & Cookies	Scitec nutrition	3.600000	80.000000	1.400000	NaN	96.599998	<NA>	a
3	Jarrow Formulas, Brown Rice Protein Concentrate, Chocolate Flavor	Confectionery	Jarrow Formulas Inc.	0.000000	70.589996	0.000000	5.900000	93.199997	<NA>	a
4	Protein chips	Chips & Crisps	Quest Nutrition Llc	0.000000	68.750000	10.940000	6.200000	90.000000	<NA>	d
5	Quest, Protein Chips, Sea Salt	Chips & Crisps	Quest Nutrition Llc	0.000000	65.620003	4.690000	0.000000	85.900002	<NA>	d
6	Quest, Protein Chips, Salt & Vinegar	Chips & Crisps	Quest Nutrition Llc	0.000000	65.620003	6.250000	0.000000	85.599998	<NA>	d
7	Thin, crunchy pretzel crackers	Crackers & Savory Snacks	Snack Factory	0.000000	53.599998	12.500000	21.400000	84.400002	1	d
8	Organic Protein Plant-Based Protein Powder - Creamy French Vanilla	Protein Snacks	Purely Inspired	5.710000	57.099998	11.400000	11.400000	81.000000	6	not-applicable
9	Turkey Creek Snacks, Pork Skins, Barbeque	Bars	Turkey Creek Pork Skins Inc.	0.000000	64.290001	35.709999	0.000000	79.400002	<NA>	e
10	Navette fleur d'oranger	Biscuits & Cookies	None	1.000000	55.000000	1.000000	NaN	77.800003	1	d
11	Baked Cheese Crisps	Chips & Crisps	Mrs. Cubbison's	0.000000	60.000000	30.000000	0.000000	77.099998	<NA>	e
12	Protein Vanilla-Kirsche	Protein Snacks	Supplement Union	2.100000	53.400002	5.100000	2.200000	76.400002	<NA>	not-applicable
13	Vegan Dream, Vegetarian Jerky, Original Hickory	Protein Snacks	Vegan Dream Inc.	3.570000	50.000000	3.570000	7.100000	75.699997	<NA>	e

Next steps: [Generate code with benchmark_df](#) [New interactive sheet](#)

9.4 - Visualise the shortlist on the nutrient matrix
 # Shows where these top products sit relative to the rest of the market

```
fig, ax = plt.subplots(figsize=(11, 7))

# Background: all products
ax.scatter(
    df["sugars_100g"], df["proteins_100g"],
    alpha=0.12, s=12, color="#adb5bd", label="All products"
)

# Highlight: full target zone
target = df[df["in_target_zone"]]
ax.scatter(
    target["sugars_100g"], target["proteins_100g"],
    alpha=0.4, s=20, color="#2a9d8f", label="Target zone products"
)

# Spotlight: top 20 benchmark shortlist
top20 = df[df["in_target_zone"]].nlargest(20, "health_score_norm")
ax.scatter(
    top20["sugars_100g"], top20["proteins_100g"],
    alpha=1.0, s=80, color="#e76f51",
    edgecolors="white", linewidths=0.8,
    zorder=5, label="Top 20 benchmark products"
)

# Label the top 5 by name
top5 = top20.head(5)
for _, row in top5.iterrows():
    name = str(row["product_name_clean"])[1:-25]
```

```

name = row["product_name_clean_17125"]
ax.annotate(
    name,
    (row["sugars_100g"], row["proteins_100g"]),
    textcoords="offset points",
    xytext=(6, 4),
    fontsize=7,
    color="#264653"
)

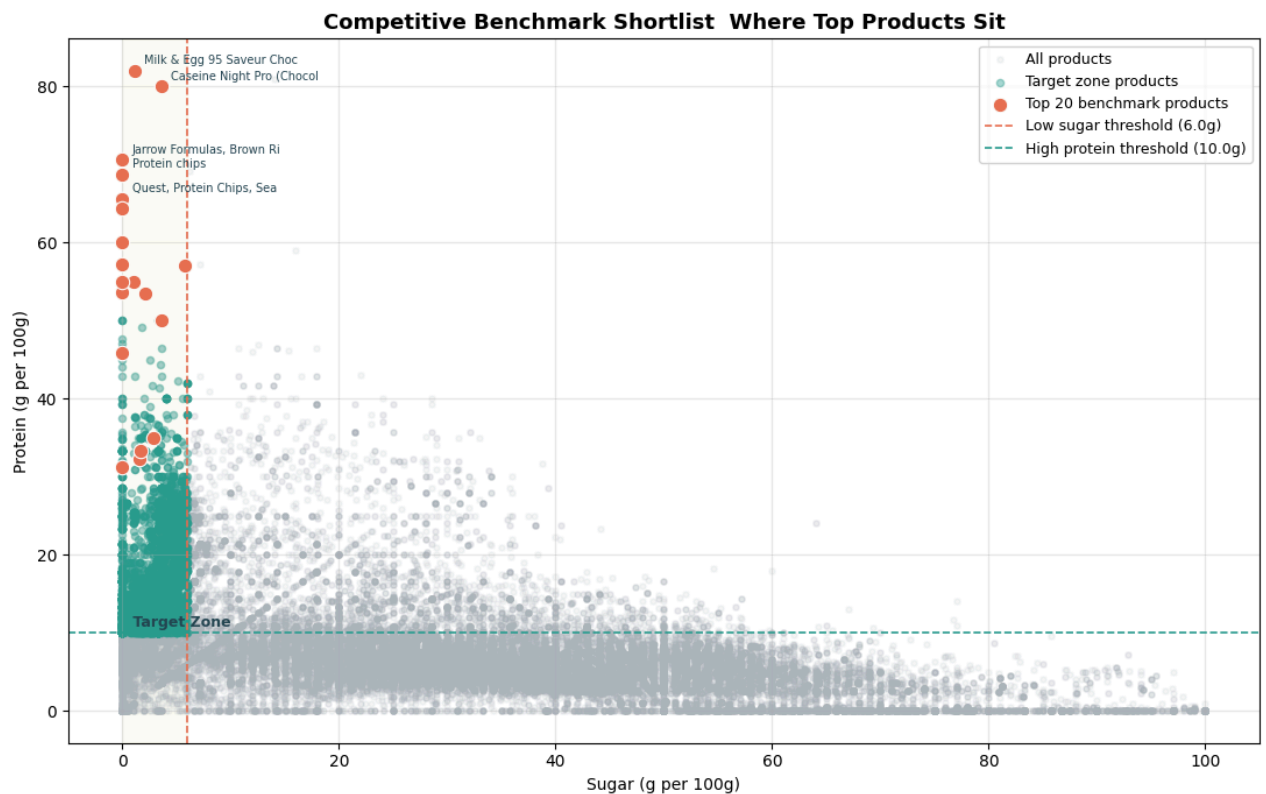
_add_quadrant_lines(ax)

ax.set_title(
    "Competitive Benchmark Shortlist Where Top Products Sit",
    fontsize=13, fontweight="bold"
)
ax.set_xlabel("Sugar (g per 100g)")
ax.set_ylabel("Protein (g per 100g)")
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(
    f"\nKey takeaway: These {len(top20)} products demonstrate that the target zone "
    f"is achievable – they already exist on shelf. The opportunity is to build "
    f"a version with stronger brand positioning and wider distribution."
)

```



10. Export Final Dataset for Dashboard

This final section saves the cleaned and enriched dataset so it can be used directly in Streamlit or another dashboard tool.